Architecture and Trade-offs

In-memory account ledger · Nitin Maurya · 1 November 2026 · describes the code at tag `v1.1-implementation`

Everything below comes from the ledger as I built it. Where I quote a number, I got it by running that code, not by estimating.

1. Append-only at scale

The ledger keeps two lists and only ever appends to them: the entries (money that moved) and the request log (every request with its outcome, refusals included). Nothing else is stored. Closing balances, available balance, authorization status, fees and interest are all recomputed from those lists, by a full scan, every time someone asks. That is why I trust it: there is no second copy of the truth that can drift. It is also the thing that breaks.

What breaks first: CPU, long before memory

After every request, the ledger re-checks every closed day of every AED account for a missing overdraft fee, and each of those checks re-scans every entry. One request therefore costs roughly *accounts* × *closed days* × *entries*. Days stay at six, but accounts and entries both grow with volume, and it happens once per request, so the whole run grows like *requests* × *accounts* × *entries*. I timed the replay on a synthetic stream with the brief's proportions, scaled by k (2k accounts, about 12k requests over the six days):

Volume	Accounts	Requests	Entries	Replay time
1×	2	18	25	22 ms (JVM warm-up)
10×	20	126	210	19 ms
100×	200	1,206	2,040	3.4 s
300×	600	3,606	6,206	96 s

Apple M2 Pro, JDK 25, one run each. The program is `docs/bench/Scale.java`; its header says how to run it.

Going from 100× to 300× made it 28 times slower, which is the cubic growth the loop predicts ($3^3 = 27$). At 300× the data is a few thousand small records, so memory is irrelevant; at 1,000× the same six days would take about an hour. The second hot spot is authorizations: their status is derived from the log, and each derivation searches the log again for a matching settlement, so a single card request is quadratic in the size of the log. In production that would show up as card authorization latency, at the p99, long before anyone noticed the batch replay.

Where state grows without bound

- **Entries and the request log** grow forever, refusals included. That is right for audit, but all of it sits in memory and all of it is scanned.
- **Holds never expire.** An authorization that is never settled stays active, so it is part of every available-balance calculation for the life of the account.
- **The fee re-check always starts from Day 1.** With a fixed six-day window that is harmless. In a bank the days never stop, so every request would re-check the account's entire history.
- The lookups for "is there already a fee / refund / reversal for this?" are full scans as well.

The cheapest structural change

Keep the log as the only source of truth, and maintain a derived index beside it, updated on every append: per account, a running total for each value date, and a marker for days that have a fee or a refund. A day's closing balance becomes a sum over days instead of a scan over entries. More importantly, the re-check after a request only needs the account it touched, from that request's value date onwards, because an entry dated Day 2 cannot change Day 1, or anybody else's account. That takes one request from *accounts* × *days* × *entries* down to a short loop over days, however many entries or accounts exist. The index is disposable: it can always be rebuilt by replaying the log, so the append-only property and "store facts, derive state" both survive.

That fixes time per request but not growth over time. The next step is a period close: once a period is signed off, snapshot every account's closing balance and stop accepting value dates inside it. Corrections go into the open period as adjustments that point at the original. That bounds the re-check, bounds how much of the log has to stay hot, and lets closed periods move to cold storage. If this were persisted, an insert-only ledger also suits PostgreSQL well: with no updates there are almost no dead row versions, so vacuum has little to do beyond freezing old rows, and partitioning by period handles the growth that remains.

2. Value-dated entries in production

In my ledger a value-dated entry is one new line, and the past simply recomputes. The replay shows what that means in practice: Day 5 closed at −230.00 with three overdraft fees on the books, and after the reversal the same day reads 465.00. Both figures were true, at different moments. My report prints two views for exactly this reason, one as each day closed and one as each day ends up. A bank has to operate with both.

Operational surface

- **Figures change after the customer has seen them.** Balances in the app, end-of-day positions and statements were all produced from the old numbers. The bank needs corrected statements and a record of what was shown to whom and when, which means keeping booking time and value time side by side (bitemporal), not just the latest answer.
- **Decisions taken on the old numbers cannot be recomputed.** Auth-B was declined because of the mistaken debit. The fees can be refunded; the decline at the till cannot be undone. Value dating repairs balances, not consequences, and those cases need complaint handling.
- **Derived amounts cascade.** One back-dated debit produced three fees on days the bank had already closed, and a fee can itself push the next day below zero. Interest also depends on which balance you use: 1.03 on the corrected balances, 0.84 if each night's figure had been frozen, because Day 5 closed at −230.00 and would have earned nothing. Every fee and interest calculation has to be re-runnable for past days and must post its difference as a new, visible entry.
- **Reconciliation and period close.** The general ledger is reconciled per date and month-end is closed. A value date inside a closed month changes figures that finance already signed off, so back-dating has to be limited to the open period, with older corrections posted as current adjustments.
- **Calendar and cut-offs.** Real value dates follow business days and payment-system cut-offs. Mine are plain numbers, so none of that is enforced (see section 4).

Regulatory surface in the UAE

- **Consumer protection.** The CBUAE Consumer Protection Standards expect charges caused by the bank's own error to be refunded immediately, and the bank must not benefit from the error. That is why my ledger refunds each fee dated on the fee's own day, so the interest the fee cost also comes back. In production the refund has to depend on why the entry was reversed: a bank error is refunded, a cancellation the customer asked for may not be.
- **Interest on credit.** My ledger charges no interest on an overdrawn balance, but a real one would: an overdraft is retail credit, and CBUAE rules (Regulation 29/2011 and its clarifications) require interest on the reducing balance. A back-dated debit then changes a regulated calculation, not just a number on a screen, and the correction must be traceable.
- **Audit trail.** Every back-dated entry has to show who entered it, who approved it, when, and why. These are exactly the entries internal audit and examiners sample, and back-dating is a well-known way to hide a movement across a reporting date, so it is also a signal for AML monitoring.
- **Reports already filed.** Regulatory returns are produced for a date. A back-dated entry can make a submitted return wrong, so the bank needs a materiality rule for restating it.

The one control I would add before going live: maker-checker on back-dated entries

Any entry whose value date is earlier than its booking day is refused unless it carries a reason code from a fixed list, an approver who is not the person who entered it, and a value date inside the open accounting period. My ledger has none of this: one caller can rewrite days that are already closed. That is the test I left failing on purpose (`KnownGapBackDatedDebitApprovalTest`), and its comments explain why I did not build it: the brief's requests carry no approver, so requiring one would reject the back-dated debit and break the replay. The reason code would also give the fee-refund decision what it is missing today, the difference between a bank error and a customer's request.

3. Authorization lifecycle

In my model an authorization is never stored. Its status (active, settled or declined) is derived from the request log each time. These are all the ways one can end, or fail to end, other than a matching settlement:

How it ends in my model	What my ledger does today	Real-world scenario	Behaviour I would mandate
Declined: not enough available	Refused with `INSUFFICIENT_AVAILABLE`, shown as DECLINED, no hold placed. Auth-B ends this way.	Card declined at the till or online.	Keep this. Add a specific decline code to the card network and a customer notification. Count an arranged overdraft limit as available, which my model has no notion of.

Refused for a structural reason	Unknown account, other currency or a date outside the window are refused and also show as DECLINED, indistinguishable from a funds decline.	Closed or blocked card; a purchase in a foreign currency; a malformed message.	Separate these from "insufficient funds": the merchant and the customer act differently on each. A foreign-currency purchase should be converted and authorized, not refused.
Duplicate ID	Resending the same request is refused (`DUPLICATE_EVENT_ID`) and places no second hold. A new request reusing the authorization ID is refused (`DUPLICATE_AUTHORIZATION`); a declined ID is burned the same way.	The acquirer times out and resends; a network delivers the message twice.	For a resend, return the original decision rather than a refusal, so the terminal gets a consistent answer. Same ID with different details: refuse and alert.
Still open when the window ends	Stays ACTIVE forever and its hold keeps reducing available: an unsettled 300.00 hold leaves available at 702.40 against a ledger balance of 1,002.40 after Day 6.	The merchant never captures: a cancelled order, an unused hotel or car-rental deposit, a fuel pre-authorization.	Expiry by merchant category in line with scheme rules, recorded as a new EXPIRED event rather than an edit, releasing the hold. A settlement arriving after expiry is a late presentment: the guarantee has lapsed, so post it if funds allow, otherwise route it to exceptions, where the bank keeps its chargeback right.
Cancelled before settlement	Cannot be expressed. A reversal only targets ledger entries and an authorization has none, so it is refused with `UNKNOWN_ORIGINAL` and the hold stays.	The merchant voids the sale; the customer cancels before dispatch.	A void request that releases the hold immediately, plus a partial void for amounts adjusted downwards.
A settlement that does not fit	More than the hold (`SETTLEMENT_EXCEEDS_HOLD`) or from another account (`UNKNOWN_AUTHORIZATION`) is refused; the authorization stays ACTIVE with its hold.	A tip or fuel amount above the authorized figure; a clearing record routed to the wrong account.	A configured tolerance per merchant category; above it, route to an exceptions queue rather than refusing silently.

Two related cases. Auth-A ends with a matching settlement for less than the hold (185.00 of 200.00) and the rest is released; a second capture is refused as `ALREADY_SETTLED`, so split shipments cannot be modelled, and production needs multi-capture up to the hold with a final-capture flag. The mirror case is a settlement with no authorization at all (Auth-Z). My ledger refuses it and moves no money, but in a real card scheme the refusal does not stop money leaving the bank: the clearing record normally arrives with the amount already owed to the scheme. Forced and offline presentments that scheme rules allow should be posted, and the rest routed to exceptions and chargebacks, never just dropped.

One behaviour I would keep as it is: a hold survives a later debit, and its settlement is accepted even when available has gone negative (900.00 settles with available at −700.00). The merchant was promised that money at authorization time. The figures in this section come from `docs/bench/Probe.java` or from the tests.

4. What I cut and why

Cut	Why it was out of scope	Production risk it defers
Concurrency	One caller, one thread.	Two authorizations racing for the same available balance could both be approved. Writes need to be serialized per account.
Persistence	The brief rules out a database.	A restart loses everything. The append-only design maps onto an insert-only table without changing the model.
Maker-checker and reason codes	The brief's requests carry no approver or reason.	One person can rewrite closed days (the failing test). Fees are refunded on every reversal that clears a day, including ones a customer asked for.
Overdraft limit on plain debits	The brief sets no limit, and its back-dated debit has to be accepted.	Any debit is accepted however large: a 1,000,000.00 debit takes an account to −999,000.00. Debits ignore holds too.
Date checks	Not needed for the brief's requests; the late instalment credit relies on the last one.	A settlement dated before its authorization is accepted, and so is a reversal dated before its original, which can create overdrafts and fees that never happened. A request keeps the booking day it states even when that day has already closed.

Hold expiry, void, multi-capture	Not in the brief (section 3).	Customers' money stays locked by holds that will never settle.
Calendar	The brief only has Day 1 to Day 6.	No weekends, holidays, cut-off times or day-count basis, all of which decide real value dates and interest.
One six-day window	That is the brief's world.	No statement cycle or period close; interest is capitalized once; the fee re-check grows with history (section 1).
Foreign exchange	No rates given; mixing currencies would invent one.	Requests in another currency are refused, but real cards are used abroad.
Fee and interest model	The brief gives one flat fee and one rate.	Fees exist only in AED, with no tariff, cap or waiver; overdrawn balances are not charged interest.
Double-entry	Only customer accounts exist.	Fee income and interest expense are never posted to the bank's own accounts, so the ledger cannot be reconciled to a general ledger; money is only checked to add up per account.
Indexes and snapshots	Full scans keep every number easy to verify.	Cubic growth per request (section 1).

I cut these to keep the core small enough that every number in it can be checked by hand, and to put the effort into what the brief actually tests: value dating, fees and interest. Three gaps that started on this list are now fixed in the code: a resent request is refused instead of applied twice, money value-dated in the future is not available to spend, and nothing is accepted once Day 6 has closed and interest is credited.